

Problem 2.1 (25 %)

In the following, we will examine two variants of gradient descent that vary in regards to how they traverse the training data set $T_{train} = \{\mathbf{x}_{(i)}, y_{(i)}\}_{i=0}^{N-1}$ throughout the iterations $k = [1, \dots, K]$.

Batch-Gradient-Descent: In BGD the loss is evaluated over the complete training set, before the weights are updated. As such, the update equation is given by (2.1.1), where $L(\mathbf{w}, \hat{y}_{(i)}, y_{(i)})$ denotes the sample-wise loss function.

$$\hat{\mathbf{w}}^{(k)} \leftarrow \hat{\mathbf{w}}^{(k-1)} - \alpha \cdot \nabla_{\mathbf{w}} J(\hat{\mathbf{w}}^{(k-1)}) \quad J(\mathbf{w}) = \frac{1}{N} \sum_{i=0}^{N-1} L(\mathbf{w}, \hat{y}_{(i)}, y_{(i)}) \quad (2.1.1)$$

Stochastic-Gradient-Descent: Here, the gradients are evaluated on a per-sample basis. Correspondingly, the weights are updated immediately. See Eq. (2.1.2) — where $i = (k-1) \bmod N$.

$$\hat{\mathbf{w}}^{(k)} \leftarrow \hat{\mathbf{w}}^{(k-1)} - \alpha \cdot \nabla_{\mathbf{w}} L(\hat{\mathbf{w}}^{(k-1)}, \hat{y}_{(i)}, y_{(i)}) \quad (2.1.2)$$

For the following task, consider a single neuron with sigmoid activation in a binary classification setup with classes $\mathcal{C}_0, \mathcal{C}_1 = \mathcal{C}_0^c$. Given the input vector $\mathbf{x} = [x_0, x_1]^T$, the output for sample s is computed through a sigmoid function $\sigma(\cdot)$ as:

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}. \quad (2.1.3)$$

Moreover, we select binary-crossentropy with l_2 regularization (2.1.4) as the loss function, where the targets $y_{(i)}$ are given by the class probabilities according to $y_{(i)} = P\{s_{(i)} \in \mathcal{C}_1\}$.

$$L(\mathbf{w}, \hat{y}_{(i)}, y_{(i)}) = -(y_{(i)} \cdot \log(\hat{y}_{(i)}) + (1 - y_{(i)}) \cdot \log(1 - \hat{y}_{(i)})) + \lambda \|\mathbf{w}\|^2 \quad (2.1.4)$$

2.1.1 Download the dataset *data.csv* and generate a surface plot of the loss function $J(\mathbf{w})$, with $L(\mathbf{w})$ from Eq. (2.1.4). Briefly compare the plots for $\lambda = 0$ and $\lambda = 10^{-2}$. *Hint: Consider using logarithmic scale.*

2.1.2 Derive an analytical expression for the gradient $\nabla_{\mathbf{w}} L(\mathbf{w}, \hat{y}_{(i)}, y_{(i)})$ of the loss function for binary-crossentropy given in (2.1.4). *Hint: Note, that the derivative of the sigmoid function is given by $\sigma(\cdot)' = \sigma(\cdot)(1 - \sigma(\cdot))$.*

2.1.3 Use the result from Task 2.1.2 to implement BGD for the regressor from (2.1.4) and fit it to *data.csv*. Plot $J(\mathbf{w})$ over the iterations $k = [1, \dots, K]$. Additionally provide a 2D scatter plot showcasing the history of the weight estimates $\hat{\mathbf{w}}^{(k)}$ throughout the optimization process. *Hint: Use the following configuration: $K = 1000$, $\alpha = 10^{-1}$, $\lambda = 10^{-2}$, $\hat{\mathbf{w}}^{(0)} = [4, -7]^T$.*

2.1.4 Repeat the Task 2.1.3 for SGD using the same configuration. What do you observe when comparing the plots for BGD and SGD? Further, specify the number of *epochs* —

denoting the a full iteration over the training dataset — required for BGD and SGD in this example. Which gradient descent variant would you choose for a large training dataset? Explain your answer.

2.1.5 Libraries for high-performance numerical computing such as *JAX* enable efficient implementation of gradient-based machine learning algorithms by providing *automatic differentiation* and just-in-time compilation. Study the tutorial in <https://docs.jax.dev/en/latest/automatic-differentiation.html> and adapt your code to compute gradients using `jax.grad` (or `jax.value_and_grad`) instead of manual derivatives. Visualize the optimization process for *BGD* and validate that the results are equivalent to the manual implementation from Task 2.1.3.

Problem 2.2 (25 %)

SVCs¹ are obtained by adjusting the loss function of the perceptron to include a buffer zone between the separating hyperplane $\mathbf{x}_{(i)}^T \mathbf{w} + b = 0$ and the symmetric translated versions of itself given by $\mathbf{x}_{(i)}^T \mathbf{w} + b = \pm 1$. This leads to the following formulation of the classification objective for a given dataset $T = \{\mathbf{x}_{(i)}, y_{(i)}\}_{i=1}^N$:

$$\begin{aligned} \mathbf{x}_{(i)}^T \mathbf{w} + b &\geq 1 && \text{if } y_{(i)} = 1 \\ \mathbf{x}_{(i)}^T \mathbf{w} + b &\leq -1 && \text{if } y_{(i)} = -1. \end{aligned} \quad (2.2.1)$$

This criterion is reflected in the SVC loss function:

$$J(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_{(i)}(\mathbf{x}_{(i)}^T \mathbf{w} + b)) + \lambda \|\mathbf{w}\|^2 \quad (2.2.2)$$

where $\lambda \geq 0$ is a hyperparameter. The term $\lambda \|\mathbf{w}\|^2$ in Eq. (2.2.2) encodes the objective, that the margin around the hyperplane should be maximized.

SVCs are of particular interest because they offer an efficient extension to a non-linear regime. By using the kernel trick, input samples can be mapped to a higher dimension $d > d'$, in which the data is again linearly separable.

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, \quad \mathbf{x}_{(i)} \mapsto \phi(\mathbf{x}_{(i)}) \quad (2.2.3)$$

2.2.1 Visualize the SVC loss function from (2.2.2) for a single sample over the soft label predictions $\hat{y} = \mathbf{x}^T \mathbf{w} + b$ for $\lambda = 0$. Distinguish the two cases $y = \pm 1$ and discuss how the SVC objective differs from the Perceptron introduced in Problem 1.4?

2.2.2 Download the file *blobs.csv* from TUWEL and generate a scatter plot that highlights the two classes in color. Fit a SVC² with a linear kernel and $C = 1$ to the dataset and extract the weights $\mathbf{w}$ and the bias b from the trained model.

2.2.3 Plot the separating hyperplane as well as the two equidistant translations running through the support vectors ontop of the scatter plot from Task 2.2.2. Calculate the distance of the margin of the separating hyperplane. *Hint: The margin distance is a function of $\mathbf{w}$ and can be derived from the geometry of the buffer area bounded by $\mathbf{x}_{(i)}^T \mathbf{w} + b = \pm 1$.*

2.2.4 Repeat Tasks 2.2.2 and 2.2.3 for the dataset *circles.csv*. Is this dataset linearly separable in the given space?

¹Support Vector Classifiers

²`sklearn.svm.SVC(C=1, kernel='linear')`

2.2.5 Find a transformation ϕ so that the *circles.csv* dataset is linearly separable in the new space $\mathbf{x}'$.

$$\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3, \quad \phi(\mathbf{x}) = \mathbf{x}' = [x_0, x_1, x_2]^T \quad (2.2.4)$$

Repeat the training of the SVC on the transformed dataset $\mathbf{x}'$ using a linear kernel and $C = 100$. Provide a 2D scatterplot of the transformed data, showcasing only x_0 and x_2' . Additionally, obtain the hyperplane and the two equidistant translations of your trained model and plot them on top of the scatterplot. *Hint: Set $x_1 = 0$ for the visual representation of the hyperplane.*

2.2.6 Now fit a SVC with a polynomial kernel³ of degree 2 to the original 2D *circles.csv* dataset and plot a heatmap of the predicted labels $\hat{y}$ for the interval $x_0 \in [-2, 2]$ and $x_1 \in [-2, 2]$. Is the separating hyperplane a linear function of $\mathbf{x}$?

2.2.7 In the dual formulation of the SVC loss function, the input data $\mathbf{x}$ only enters in the form of the inner product $\langle \mathbf{x}_{(i)}, \mathbf{x}_{(j)} \rangle$ between two samples i and j . As such, the transformation $\phi(\mathbf{x})$ is implicitly defined via the kernel function k :

$$k(\mathbf{x}_{(i)}, \mathbf{x}_{(j)}) = \langle \phi(\mathbf{x}_{(i)}), \phi(\mathbf{x}_{(j)}) \rangle. \quad (2.2.5)$$

The polynomial kernel of degree 2 used in Task 2.2.6 is for instance given by:

$$k_{\text{poly}, d=2}(\mathbf{x}_{(i)}, \mathbf{x}_{(j)}) = (\mathbf{x}_{(i)}^\top \mathbf{x}_{(j)} + 1)^2. \quad (2.2.6)$$

Find an expression for the transformation $\mathbf{x}' = \phi(\mathbf{x})$ that is implicitly applied to $\mathbf{x} = [x_0, x_1]^T$ via the polynomial kernel in Equation (2.2.6). What is the dimension d' after the transformation?

³sklearn.svm.SVC(C=1, kernel="poly", degree=2)

Problem 2.3 (25 %)

Let us revisit *Ridge Regression* from Exercise 1.1 with the loss function given by:

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \frac{1}{2} \lambda \|\mathbf{w}\|^2. \quad (2.3.1)$$

Here, we used the pseudoinverse to compute the weight vector $\hat{\mathbf{w}}$ which allows us to obtain predictions for new data samples $\mathbf{X}'$ according to:

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_{d \times d})^{-1} \mathbf{X}^T \mathbf{y} \quad \hat{\mathbf{y}} = \mathbf{X}' \hat{\mathbf{w}} = \mathbf{X}' (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_{d \times d})^{-1} \mathbf{X}^T \mathbf{y}. \quad (2.3.2)$$

In the following we will derive an adapted version of *Ridge Regression*, namely *Kernel Ridge*, which utilizes the Kernel trick to obtain a non-linear variant of the regression scheme. Equivalently to the SVM, we apply the transformation $\mathbf{x}_{(i)} \rightarrow \phi_{(i)} = \phi(\mathbf{x}_{(i)})$, which maps the input from $\mathcal{R}^d$ to $\mathcal{R}^{d'}$.

The essence of the Kernel trick is to not explicitly compute the transformed inputs $\phi_{(i)}$, but to derive a formulation where the mapping $\phi(\cdot)$ is implicitly defined via the kernel $k(\mathbf{x}_{(i)}, \mathbf{x}_{(j)}) = \langle \phi(\mathbf{x}_{(i)}), \phi(\mathbf{x}_{(j)}) \rangle$.

2.3.1 Show that the inverse in (2.3.2) can equivalently be applied in the data domain with dimension n instead of the feature domain with dimension d . For that, prove that the following matrix identity holds:

$$(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_{d \times d})^{-1} \mathbf{X}^T = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}_{n \times n})^{-1}. \quad (2.3.3)$$

2.3.2 Now, use the above matrix identity to obtain a variant of Equation 2.3.2, where the input vectors $\mathbf{x}_{(i)}$ in $\mathbf{X}$ only enter via the inner product. Further, implement the corresponding estimator in Python for arbitrary degrees m of the polynomial kernel:

$$k(\mathbf{x}_{(i)}, \mathbf{x}_{(j)}) = (\mathbf{x}_{(i)}^\top \mathbf{x}_{(j)} + 1)^m. \quad (2.3.4)$$

2.3.3 Fit your regressor to the 1D dataset from Exercise 1.1 using $m \in [1, 2, 3, 10]$ and plot the predictions in the interval $x \in [0, 2]$. Also make sure to analyse the influence of different regularization parameters.

2.3.4 Repeat Task 2.3.3 with the RBF kernel given by $k(\mathbf{x}_{(i)}, \mathbf{x}_{(j)}) = e^{-\frac{1}{2l_s^2} \|\mathbf{x}_i - \mathbf{x}_j\|^2}$. Briefly discuss your interpretation of the role of the kernel parameter l_s .

2.3.5 Derive an analytical expression of the mapping $\mathbf{x}_i \rightarrow \phi_i = \phi(\mathbf{x}_i)$ implied by the RBF kernel for the scalar case $d = 1$. What is the dimension d' after the transformation? *Hint: Use the series expansion of the exponential function $e^x = \sum_{i=0}^{\infty} \frac{x^n}{n!}$.*

Problem 2.4 (25 %)

When performing classification tasks, we are often interested not only in predicting the actual class labels but also in obtaining reliable probability estimates for each class. A well-calibrated classifier produces probability estimates that are, on aggregate, closely aligned with the actual outcomes. This means that, for binary classification, if we predict a probability of 0.8 for the positive class, then approximately 80% of the samples with this predicted probability should actually belong to the positive class.

2.4.1 Download the file *calibration.csv* from TUWEL and use *pandas* to extract the data. Split the dataset into $T_{\text{train_full}}$ and T_{test} using a test size of 0.2. For that, use *sklearn.model_selection.train_test_split* and set the *random_state* to 256. Subsequently, split $T_{\text{train_full}}$ into T_{train} and T_{cal} using a *test_size* of 0.2 and set the *random_state* to 256.

2.4.2 Fit a random forest classifier⁴ from *scikit-learn* to the training dataset T_{train} . What is the accuracy of your model on T_{test} ?

Calculate⁵ and plot⁶ the confusion matrix — see Table 2.4.1 — of your classifier on T_{test} and provide the values for *Precision*, *Recall*, and *F1-Score*. *Hint: The Precision is given by $\frac{TP}{TP+FP}$, while the Recall is defined as $\frac{TP}{TP+FN}$. We denote the harmonic mean of both measures as the F1-Score.*

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	True Negatives (TN)	False Positives (FP)
$y = 1$	False Negatives (FN)	True Positives (TP)

Table 2.4.1: Confusion Matrix

The most common way to assess model calibration is plotting the *reliability diagram* often referred to as *calibration curve*. It shows the fraction of true positives over the predicted probabilities of the positive class. Thus, the calibration plot of a perfectly calibrated classifier corresponds to a 45-degree line. You will next learn how to manually calculate and plot the *reliability diagram*.

2.4.3 First, predict the class probabilities on T_{test} using the method *RandomForestClassifier.predict_proba()*. Next, create a *pandas* dataframe that contains the true labels and the predicted probabilities of the positive class (second column resulting from *predict_proba*) of the dataset T_{test} . Add a column that tells us which bin each sample belongs to. For that, create a numpy array containing the bin boundaries with *np.linspace(0,1,11)* and use *pandas.cut()* with respect to the predicted probabilities.

⁴`sklearn.ensemble.RandomForestClassifier(n_estimators=500, random_state=256)`

⁵`sklearn.metrics.confusion_matrix()`

⁶`sklearn.metrics.ConfusionMatrixDisplay()`

The x values of the *reliability diagram* are given by the mean predicted probability of each bin and the y values are given by the fraction of positives in each bin. *Hint: Provided the classes are labeled with 0 and 1, the fraction of positives corresponds to the mean of the true class labels. Consider using `pandas.groupby()` for calculating x and y.*

Plot the *reliability diagram* of your uncalibrated classifier and add the calibration curve of a perfectly calibrated classifier to your plot. Is your classifier well calibrated? Is it over/underpredicting?

In addition, compute the log-loss⁷, also known as *binary-crossentropy*, on the test dataset T_{test} . Although it assesses the combination of calibration and discrimination, it is frequently used as a calibration "metric". Note that, besides the fact that smaller is better, the log-loss is difficult to interpret on its own. However, it can be used comparatively.

2.4.4 *Scikit-learn* contains several useful classes/methods for calculation and visualization of the *reliability diagram*.

Calculate and plot the *reliability diagram* in one step using the method `from_estimator()` of `sklearn.calibration.CalibrationDisplay()`. Don't forget to set the `n_bins` to 10. Compare the plot with the *reliability diagram* from 2.4.3.

Next, you will learn how to calibrate your random forest classifier. Probability calibration is a post-processing task, which consists of fitting a regressor that maps the output of the classifier, i.e., the predicted probability, to a calibrated probability in $[0,1]$. Ideally, it is fit on a dataset which is independent of the training data used to fit the classifier.

2.4.5 In order to calibrate the random forest classifier, we will use a technique called *Platt scaling* which is given as

$$P(y_{(i)} = 1 | f_{(i)}) = \frac{1}{1 + \exp(a f_{(i)} + b)}. \quad (2.4.1)$$

Here, $y_{(i)}$ denotes the true class label and $f_{(i)}$ the predicted probability output of the uncalibrated classifier for sample i . Note that *Platt scaling* corresponds to logistic regression without the regularization (penalty) term.

We use *scikit-learn's* class `sklearn.calibration.CalibratedClassifierCV()` to perform *Platt scaling*. Create an instance of `sklearn.calibration.CalibratedClassifierCV()`, set `cv="prefit"` and `method="sigmoid"`. Fit the object on T_{cal} and predict the probabilities on T_{test} . Derive the class labels from the predicted probabilities, calculate the calibrated classifier's accuracy and compare it with your uncalibrated classifier from 2.4.2.

Finally, calculate the log-loss and visualize the *reliability diagram* of the calibrated classifier. Compare your results with the uncalibrated classifier and briefly interpret your

⁷`sklearn.metrics.log_loss()`

findings.

Additional Information:

Besides *Platt scaling*, another frequently used calibration technique is *isotonic regression*. The interested participant is invited to read the corresponding scikit-learn documentation⁸ and repeat Task 2.4.5 using `sklearn.CalibratedClassifierCV()` with `method="isotonic"`.

Furthermore, the package *ml-insights* contains an additional calibration method and some very useful tools for assessing model calibration. For further information, check the documentation⁹ and examples.

⁸<https://scikit-learn.org/stable/modules/calibration.html>

⁹<https://ml-insights.readthedocs.io/en/latest/>